MindAi

NestJS Application -- Full Codebase Reference

AI-powered analytics platform with conversational memory

Built on NestJS * MongoDB / Mongoose * Groq LLM * Angular 21

Covers every file: architecture, request flow,
AI pipeline, caching, memory, and all modules.

NestJS | MongoDB | Groq LLM | Angular 21

CHAPTER 1

Project Overview

What MindAi is and why a NestJS app exists

What is MindAi?

MindAi is an AI-powered analytics platform. Users type a natural-language question -- "Show me project spending by municipality" -- and the system automatically:

1. Calls a Groq LLM (Supervisor) to translate the question into a MongoDB aggregation pipeline
2. Runs that pipeline against the configured data collections
3. Calls a second LLM (Chart or Writer) to format the results as a dashboard, report, or summary
4. Returns structured output rendered by the Angular frontend using ECharts

Why Two Backends?

The repository contains two separate backend implementations that do the same job:

src/ Express backend (original) -- runs on port 3000
nest-app/ NestJS backend (structured rewrite) -- runs on port 3001

The NestJS version is the more production-ready implementation. It adds:

* NestJS dependency injection (DI) -- services, modules, guards, filters wired automatically
* Mongoose ODM -- schema-first MongoDB access with @Schema / @Prop decorators
* Long-term memory extraction -- after every response, key facts are extracted and stored
* Embedding-based semantic retrieval -- memories recalled by meaning, not keywords
* API key fallback -- if a user's key fails, transparently retries with the global key

Technology Stack

Layer	Technology	Purpose
`Runtime`	Node.js 22 / TypeScript	Server language
`Framework`	NestJS 10	DI, modules, decorators, lifecycle
`HTTP`	Express (under NestJS)	HTTP transport
`Database`	MongoDB + Mongoose	Persistent storage + ODM
`Session Memory`	LibSQL / SQLite (Mastra)	Short-term conversation threads
`Long-term Mem`	MongoDB memory_items	Cross-session semantic memory
`LLM Provider`	Groq (OpenAI-compatible API)	Fast llama-3.3-70b inference
`AI SDK`	Vercel AI SDK (generateObjec	Structured LLM output + Zod
`Agent`	Mastra Agent + Tools	Free-text intent routing
`Embeddings`	@xenova/transformers (WASM)	Local embedding model, no API key

Frontend	Angular 21 + ECharts	Chart rendering, UI
Validation	class-validator + Zod	DTO & schema validation
Security	Helmet + CORS + ApiKeyGuard	HTTP security headers & auth

High-Level Architecture

Directory structure

```
nest-app/src/
  main.ts                App entry point (port 3001)
  app.module.ts          Root DI module
  config/                All env variables typed & grouped
  constants/             LLM model/timeout lookup tables
  common/                Logger, guard, filter, middleware (shared)
  database/              MongoDB connection (DatabaseModule)
  ai/                    All LLM calls + embedding model
  analytics/             Main HTTP handler (controller+service+pipeline)
  features/              Standalone feature functions (agent fallback path)
  session/               Short-term memory (LibSQL) + Mastra agent
  memory/                Long-term semantic memory (MongoDB + embeddings)
  history/               Pipeline run logs + session history
  cache/                 Prompt cache (7-day TTL, SHA-256 keyed)
  sources/               Data source registry + in-memory cache
  saved-results/         User-saved report snapshots
  user-keys/             Per-user Groq API key storage
  prompts/               System prompt builders for each LLM role
  types/                 Shared TypeScript interfaces
```

CHAPTER 2

Entry Point & Configuration

main.ts, app.module.ts, configuration.ts, Model.ts

nest-app/src/main.ts

Application bootstrap -- first file executed when the server starts

main.ts is the Node.js entry point. It performs these steps in order:

1. NestFactory.create(AppModule) -- instantiates the entire DI container and all modules
2. app.useLogger(AppLogger) -- replaces NestJS default logger with the custom colored logger
3. app.use(helmet()) -- adds security HTTP headers (CSP disabled for Angular compatibility)
4. app.enableCors() -- allows browser requests from configured ALLOWED_ORIGINS
5. app.useGlobalPipes(ValidationPipe) -- auto-validates all DTO classes on every request
6. app.enableShutdownHooks() -- listens for SIGTERM/SIGINT for graceful shutdown
7. app.listen(3001) -- starts the HTTP server
8. warmupEmbeddings() -- pre-loads the 25MB local AI model in background

The port is 3001 (not 3000) to avoid conflict with the Express backend running on 3000.

main.ts -- simplified bootstrap

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule, { bufferLogs: true });
  app.useLogger(app.get(AppLogger));
  app.use(helmet({ contentSecurityPolicy: false }));
  app.enableCors(allowedOrigins.length ? { origin: allowedOrigins } : { origin: true });
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
  app.enableShutdownHooks();
  await app.listen(3001);
  warmupEmbeddings();  // background -- no await
}
```

nest-app/src/app.module.ts

Root module -- wires every feature module + global providers

AppModule is the root of the NestJS dependency injection tree. Every feature module is imported here. Two providers are registered globally:

APP_GUARD -> ApiKeyGuard -- runs before EVERY route handler automatically
APP_FILTER -> AllExceptionsFilter -- catches EVERY uncaught exception app-wide

It also conditionally serves the Angular static files if client/dist/ exists. The RequestIdMiddleware is applied to all routes via configure().

Module	What it provides
`ConfigModule (global)`	ConfigService -- typed env variable access everywhere
`DatabaseModule`	MongoDB connection via Mongoose forRootAsync
`SourcesModule`	Data source registry, in-memory cache, /api/sources routes
`HistoryModule`	Pipeline run history, session history routes
`CacheModule`	Prompt cache (7-day TTL) + /api/cache routes
`SavedResultsModule`	User-saved analytics snapshots
`UserKeysModule`	Per-user Groq API key storage
`AnalyticsModule`	Main analytics controller + pipeline service
`ServeStaticModule`	Serves Angular build if present (optional)

nest-app/src/config/configuration.ts

All environment variables typed and grouped into a config object

Returns a typed config factory read by NestJS ConfigModule. Groups variables into three namespaces:

server.* -- PORT, SHUTDOWN_TIMEOUT_MS, ALLOWED_ORIGINS, API_KEY
mongodb.* -- URI, DB name, timeouts, retry count, pipeline timeout
llm.* -- GROQ_API_KEY, per-role timeouts, default model name

The helper positiveInt(env, fallback) safely converts string env vars to numbers, falling back to the default if the value is missing or invalid.

nest-app/src/constants/Model.ts

LLM role lookup tables -- model names, env keys, timeouts

Defines three Record<AgentRole, string> objects used by ai/model.ts:

ROLE_ENV_KEYS -- which env variable overrides the model for each role
ROLE_DEFAULTS -- default model if no override is set
TIMEOUT_ENV_KEYS -- which env variable sets the timeout for each role

The "memory" role deliberately uses a smaller, cheaper model (llama-3.1-8b-instant) because memory extraction runs on every response and does not need the full 70B model.

Role	Default Model	Env Override
`supervisor`	llama-3.3-70b-versatile	GROQ_SUPERVISOR_MODEL
`chart`	llama-3.3-70b-versatile	GROQ_CHART_MODEL
`writer`	llama-3.3-70b-versatile	GROQ_WRITER_MODEL
`memory`	llama-3.1-8b-instant	GROQ_MEMORY_MODEL

CHAPTER 3

Common Utilities

Logger, Guard, Filter, Middleware -- shared across all modules

common/logger/app.logger.ts

Custom colored console logger with per-request ID tracking

Provides two things:

1. Standalone functions (log, logTrace, logSep) -- used by plain code (repositories, services) that is not injectable. These call emit() internally.

2. AppLogger class (implements LoggerService) -- used by NestJS itself for framework messages (module startup, route registration, errors).

Both share a singleton AsyncLocalStorage store (requestContext). The RequestIdMiddleware puts a UUID into this store at the start of each HTTP request. Every log line then automatically includes a short prefix like [a3f7b2c1] so you can grep all log lines belonging to one request.

Log levels: log (info), warn, error, debug (only if DEBUG=1 env), verbose (suppressed). TRACE=1 env enables logTrace() calls which dump full JSON objects like LLM plans and results.

app.logger.ts -- simplified core

```
export const requestContext = new AsyncLocalStorage<{ requestId: string }>();

function emit(tag: string, msg: string): void {
  const store = requestContext.getStore();
  const reqId = store ? `[${store.requestId.slice(0, 8)}]` : "";
  const color = TAG_COLORS[tag] ?? COLORS.gray;
  console.log(`[${ts()}]${reqId} [${tag}] ${msg}`);
}

export function log(tag: string, msg: string): void { emit(tag, msg); }
```

common/middleware/request-id.middleware.ts

Assigns a UUID to each request via AsyncLocalStorage

An Express/NestJS middleware that runs before every route handler. Calls requestContext.run({ requestId: randomUUID() }, next) which wraps the entire async call chain in a context zone. Any code downstream -- even in async callbacks -- can call requestContext.getStore() to get the same request ID without it being passed as a parameter. This is how the logger correlates log lines to requests.

common/guards/api-key.guard.ts

Server-level authentication -- validates x-api-key header

Implements CanActivate. Applied globally via APP_GUARD in app.module.ts, so it runs on every single route.

Logic:
1. If API_KEY env var is not set -> pass all requests through (dev mode)
2. If the requested path is in the public list -> pass through
3. Otherwise -> check x-api-key header matches API_KEY -> throw 401 if wrong

Public routes (no auth required):
/api/meta, /health, /api/provider -- read-only metadata
/api/key/* -- user API key management (uses x-user-id instead)

`common/filters/all-exceptions.filter.ts`

Global error handler -- converts any exception to consistent JSON

Catches every exception thrown anywhere in the app (decorated with @Catch() which matches all exception types). Applied globally via APP_FILTER in app.module.ts.

Behavior:
* HttpException (400, 401, 404 etc) -> uses its status + message
* Any other Error -> 500 + logs full stack trace
* Extracts optional "code" field (e.g. INVALID_API_KEY, LLM_RATE_LIMIT) and includes it in the response so the frontend can display specific error messages

Response shape: { error: "message", code?: "SOME_CODE" }

CHAPTER 4

Database Layer

MongoDB connection, Mongoose schemas, raw MongoClient

Two Database Clients

The nest-app uses MongoDB in two ways simultaneously:

1. Mongoose (via MongooseModule) -- the primary ORM. Used by all injectable services. Registered in DatabaseModule and available via @InjectModel() and @InjectConnection() decorators.

2. Raw MongoClient (mongo.client.ts) -- a standalone singleton. Used only by the Mastra agent fallback path (features/pipeline.ts) which is not inside the NestJS DI container and cannot use @InjectConnection().

Both point to the same MongoDB database specified by MONGODB_URI + MONGODB_DB env vars.

`database/database.module.ts`

Sets up Mongoose global connection using NestJS async factory

Uses MongooseModule.forRootAsync() with a ConfigService factory. This pattern defers the connection until the ConfigModule is ready, ensuring env variables are loaded first. Sets serverSelectionTimeoutMS from config (default 8 seconds). Any module that imports DatabaseModule (or imports it transitively) gets access to the Mongoose connection for injecting models.

`database/mongo.client.ts`

Raw MongoClient singleton with retry logic -- used by agent fallback path

A module-level singleton (client, db variables). getMongo() is idempotent -- returns the existing connection if already established. On first call, it attempts to connect up to MONGODB_CONNECT_RETRIES times (default 1). If all attempts fail, throws a descriptive error including the database name and timeout. closeMongo() is available for graceful shutdown.

MongoDB Collections Used

Collection	Purpose
`sources`	Registered DataSource metadata (schema, fields, joins)
`prompt_cache`	7-day TTL cache keyed by SHA-256 of intent+prompt
`results_history`	Every pipeline execution log (prompt, pipeline, rows, timing)
`chart_results`	Cached dashboard specs from runChart()
`saved_results`	User-saved report/dashboard snapshots
`user_api_keys`	Per-user Groq API key (userId -> apiKey mapping)

`memory_items`	Long-term semantic memories (type, content, embedding, tags)
`Data collections`	e.g. projects, municipalities -- seeded by scripts/seed.ts

CHAPTER 5

AI Layer

LLM calls, chart rendering, embeddings, memory extraction

Overview -- LLM Roles

The AI layer makes up to 3 LLM calls per user request. Each call uses a separate "role" with its own model, timeout, and system prompt:

Role	File	Purpose
`supervisor`	ai/planner.ts	Translates prompt -> MongoDB pipeline + strategy
`chart`	ai/chart.ts	Selects chart types + field mappings
`writer`	ai/writer.ts	Writes report sections or inquiry summary
`memory`	ai/memory-skill.ts	Extracts key facts from completed turns

`ai/model.ts`

Groq client factory -- resolveModel(), freshSignal(), modelName()

Three functions used by every LLM call in the system:

resolveModel(role, apiKey) -- creates a Groq OpenAI-compatible client. The baseURL is https://api.groq.com/openai/v1. Picks the model name from ROLE_DEFAULTS or the appropriate env variable override. The apiKey parameter lets per-user keys override the global key on a per-request basis.

freshSignal(role) -- creates a new AbortSignal.timeout(ms) for the role's configured timeout. A fresh signal is created for each call so timeouts don't accumulate across retries.

modelName(role) -- returns just the model name string (used for logging).

`ai/planner.ts`

Supervisor LLM -- converts natural language to MongoDB aggregation pipeline

This is LLM Call #1. It is the most critical function in the system.

Input: user prompt + intent + list of all registered DataSource schemas + conversation context
Output: a TaskPlan with { needsData, pipeline, skills, strategy, chartHint }

The Zod schema passed to generateObject() changes based on intent:
* dashboard -- adds strategy enum + chartHint string
* report -- adds wantChart boolean + optional strategy

* inquiry -- base schema only

After the LLM responds, finalizeTaskPlan() resolves the source name to the actual registered DataSource (using normalizeToken for fuzzy matching) and calls deriveExecutionSkills() to determine which downstream skills to run:

dashboard -> [aggregation, chart]
report -> [aggregation, report] or [aggregation, report, chart] if wantChart
inquiry -> [aggregation, inquiry]

Settings: temperature=0 (deterministic), maxRetries=1, maxTokens=600.

ai/chart.ts

Chart renderer -- LLM selects widget types, deterministic code renders them

The largest file (~530 lines). Two-phase approach:

Phase 1 -- LLM: generateObject() with dashboardSchema (layout + widgets[]). The LLM picks chart types, field names for axes/values/labels, aggregation method, optional topN/sortDesc, and an insight string. This is constrained by Zod enums loaded from skills/chart/SKILL.md at startup.

Phase 2 -- Deterministic: For each proposed widget:
a) validateWidget() -- checks field names exist in actual data, numeric fields are numeric, required fields like seriesField are present
b) toWidgetPlan() -- normalizes the widget (e.g. auto-detects valueField if missing)
c) renderWidget() -> calls the specific renderer function

Renderer functions produce ECharts option objects. All 15 chart types are supported:

Chart Type	Key Fields	Notes
`bar_chart`	labelField, valueField	Vertical bars
`horizontal_bar_chart`	labelField, valueField	Flipped axes
`grouped_bar_chart`	labelField, valueField,serie	Side-by-side groups
`stacked_bar_chart`	labelField, valueField,serie	Stacked totals
`line_chart`	xField, valueField	Auto-sorts by axis
`area_chart`	xField, valueField	Line with fill
`multi_line_chart`	xField, valueField,seriesFie	Multiple series
`donut_chart`	labelField, valueField	Pie with hole
`scatter_plot`	xField, yField	Optional labelField
`kpi_card`	valueField	Single number display
`gauge_chart`	valueField	Circular gauge 0-100
`funnel_chart`	labelField, valueField	Funnel visualization
`radar_chart`	labelField, columns[]	Multi-metric spider
`heatmap`	xField, yField, valueField	Grid intensity map
`table`	columns[]	Raw data table

ai/writer.ts

Writer LLM -- generates inquiry summaries and report sections

Two exported functions:

runInquirySkill({ prompt, rows }) -- LLM Call #2 for inquiry intent. System prompt read from skills/inquiry/SKILL.md at startup. Returns { summary: string } (max 400 tokens). Sends only the first INQUIRY_MAX_ROWS rows to keep token usage low.

runReportSkill({ prompt, rows, withChart }) -- LLM Call #2 for report intent. System prompt from skills/report/SKILL.md. Returns { reportSections: [{heading, body}] } with 1-5 sections (max 900 tokens). The withChart flag tells the LLM whether a visual chart will accompany the report so it can adjust the prose accordingly.

Both use: temperature=0, maxRetries=1, generateObject() with Zod schema.

ai/memory-skill.ts

Memory extraction LLM -- pulls key facts from completed conversations

Called fire-and-forget after every analytics response (in AnalyticsService.maybeExtractMemory). Uses the cheaper "memory" role model (llama-3.1-8b-instant).

Input: user prompt + AI response summary
Output: up to 3 memory items, each with:
* type: goal | insight | preference | context | decision
* content: 5-300 character string
* tags: up to 5 keyword strings
* importance: integer 1-5

maxRetries=0 and errors are caught/ignored -- memory extraction never blocks responses. System prompt is read from skills/memory/SKILL.md.

ai/skill-prompt.ts

Reads system prompts from Markdown files in the skills/ folder

Three utility functions:

skillFile(...segments) -- resolves the absolute path to a file in skills/

readMarkdownSection(filePath, heading) -- extracts the text content under a specific ## Heading in a Markdown file, stopping at the next heading of equal or higher level. Used to load system prompts at module startup (once, not per request).

readJsonSection(filePath, heading) -- calls readMarkdownSection then parses the first ```json code block inside that section. Used by chart.ts to load the chart type definitions and aggregation lists from skills/chart/SKILL.md.

interpolateTemplate(template, values) -- replaces {{VARIABLE}} placeholders. Used for dynamic prompt assembly with

values like source name or field list.

ai/embeddings.ts

Local sentence embedding model -- converts text to 384-dimensional vectors

Uses @xenova/transformers to run the all-MiniLM-L6-v2 model locally via WebAssembly. No GPU, no API key, no external service required. Downloads ~25MB to .model-cache/ on first run.

Singleton pattern: _module, _pipe, _loading ensure the model is loaded exactly once even if multiple requests arrive before loading completes. _loading holds the in-flight promise so concurrent callers await the same load operation.

warmupEmbeddings() -- called from main.ts after server start. Pre-loads the model in background so the first real request does not wait for the download.

generateEmbedding(text) -- runs text through the pipeline with pooling:mean and normalize:true. Returns Float32Array as number[] (384 values).

cosineSimilarity(a, b) -- dot product of two L2-normalised vectors. Since normalize:true ensures |v|=1, dot product equals cosine similarity (range 0 to 1 for similar texts).

embeddings.ts -- cosineSimilarity

```
// Why dot product = cosine similarity:
// cos(a, b) = (a * b) / (|a| * |b|)
// when |a| = |b| = 1 (normalized): cos(a, b) = a * b

export function cosineSimilarity(a: number[], b: number[]): number {
  let dot = 0;
  for (let i = 0; i < a.length; i++) dot += a[i] * b[i];
  return dot;
}
```

CHAPTER 6

Analytics Module

The main request handler -- controller, service, pipeline service

analytics/analytics.controller.ts

HTTP layer -- two routes, DTO validation, user ID extraction

Routes:

GET /api/provider -- public, returns { provider: "groq", hasGlobalKey: bool }. The frontend uses hasGlobalKey to decide whether to prompt the user for their own key.

POST /api/analytics -- the main endpoint. Accepts AnalyticsDto:
* prompt (string, 1-1000 chars, required)
* intent (string, optional) -- "dashboard", "report", "inquiry", or omit for free-text
* sessionId (string, optional) -- resume an existing conversation

The x-user-id header is required on POST. requireUserId() throws 401 if missing. HttpExceptions propagate directly; other errors are wrapped in 500.

analytics/analytics.service.ts

Orchestrates the full request -- API keys, session, memory, pipeline, response

The most complex service in the app. run() method flow:

1. promptSchema.parse() -- Zod validates prompt length
2. ensureDataSources() -- throws 400 if no sources are registered
3. resolveApiKeys() -- loads user's stored key from MongoDB, falls back to global key
4. resolveSession() -- resumes session if sessionId exists in LibSQL, else creates new UUID
5. buildMemoryContext() -- loads last 6 messages + top 3 long-term memories
6. runWithFallback() -> PipelineService.execute*()
7. If execution fails with 401/403 -> retry with global key (per-user key deleted)
8. If execution fails with 429 -> retry with global key or throw rate-limit error
9. buildResponse() -- assembles HTTP response, saves turn, extracts memories (both async)

The key fallback mechanism ensures users with invalid keys still get results if a global key is configured. Per-user keys are silently deleted when they fail.

analytics.service.ts -- run() simplified

```
  // Simplified run() flow
  async run(req: AnalyticsRequest): Promise<AnalyticsResponse> {
    const prompt = promptSchema.parse(req.prompt);
```

```
    this.ensureDataSources();
    const { primaryKey } = await this.resolveApiKeys(req.userId);
    const { sessionId }  = await this.resolveSession(req);
    const memCtx = await this.buildMemoryContext(req.userId, sessionId, prompt);
    const { result } = await this.runWithFallback(req.intent, prompt, memCtx, ...);
    return this.buildResponse({ result, prompt, sessionId, ... }); // fires memory async
  }
```

analytics/pipeline.service.ts

Core AI pipeline -- aggregate() + executeDashboard/Report/Inquiry()

The NestJS injectable pipeline orchestrator. Uses @InjectConnection() for the Mongoose connection and injects CacheService, HistoryService, ChartResultsRepository.

aggregate(prompt, intent, context, apiKey):

1. CacheService.getCached() -- skip LLM if identical prompt was answered before
2. runSupervisorPlan() -- LLM Call #1 -> TaskPlan
3. resolvePipeline() -- validates pipeline safety (no $function/$merge/$out), finds collection name
4. runAggregation() -- Mongoose connection runs the pipeline with maxTimeMS guard
5. flush() -- writes to cache + history (both fire-and-forget)

executeDashboard() -> aggregate() -> runChart()
executeReport() -> aggregate() -> runReportSkill() + optional runChart() in parallel
executeInquiry() -> aggregate() -> runInquirySkill()

Dashboard also checks a "full" cache key that stores the complete rendered spec (skipping both LLM calls on cache hit).

SECURITY: Forbidden Pipeline Stages

The following MongoDB stages are blocked to prevent data exfiltration or side effects: $function, $merge, $out, $where, $eval. Any plan containing these throws an error before execution.

CHAPTER 7

Full Request Flow

Step-by-step trace of POST /api/analytics

Happy Path -- Dashboard Request

Trace of: POST /api/analytics { prompt: "show spending by city", intent: "dashboard" }

1	**RequestIdMiddleware**	Assigns UUID to AsyncLocalStorage
2	**ApiKeyGuard**	Validates x-api-key header
3	**ValidationPipe (DTO)**	Checks prompt 1-1000 chars, types
4	**AnalyticsController**	Extracts x-user-id header
5	**AnalyticsService.run()**	Starts orchestration
6	**UserKeysService.get()**	Loads user Groq API key from MongoDB
7	**sessionExists() / UUID**	Resolves or creates session ID
8	**getMemoryContext()**	Loads last 6 messages from LibSQL
9	**getRelevantContext()**	Finds top 3 semantic memories from MongoDB
10	**PipelineService**	executeDashboard() called
11	**CacheService.getCached()**	SHA-256 key lookup in MongoDB (miss)
12	**runSupervisorPlan()**	LLM #1: prompt -> MongoDB pipeline (Groq)
13	**runAggregation()**	Mongoose runs pipeline against data collection
14	**runChart()**	LLM #2: rows -> widget plans (Groq)
15	**Chart renderers**	Deterministic code builds ECharts configs
16	**CacheService.setCached()**	Saves full dashboard spec (7-day TTL)
17	**HistoryService.save()**	Logs run to results_history collection
18	**buildResponse()**	Assembles HTTP response JSON
19	**saveConversationTurn()**	Saves to LibSQL session (fire-and-forget)
20	**extractAndSave()**	LLM #3: memory extraction (fire-and-forget)

Total LLM calls: 2 blocking (#1 supervisor, #2 chart) + 1 background (#3 memory)

Free-Text Routing (no intent)

When no intent is provided, AnalyticsService calls analyticsAgent.generateLegacy() (the Mastra agent in session/agent.ts). The agent has 3 tools:

buildDashboard, generateReport, executeInquiry

The Supervisor LLM reads the prompt and the tool descriptions, then decides which tool to call. This adds one extra LLM call at the start. The agent tools call the standalone functions in features/ (not PipelineService) because Mastra tools are not inside the NestJS DI container.

Cache Hit Path (repeated prompt)

If the exact same prompt+intent was already answered and is within 7 days:

CacheService.getCached() -> cache hit -> return cached result

Zero LLM calls. Zero MongoDB aggregation. The cache key is a 24-character prefix of SHA-256("intent:normalized_prompt"). The prompt is lowercased and whitespace-normalized before hashing so minor variations still hit the cache.

CHAPTER 8

Session & Memory

Short-term conversation threads + long-term semantic memory

Two Memory Systems

System	Details
`Short-term (LibSQL)`	Last 6 messages per session, stored in SQLite file ./data/memory.
`Short-term (LibSQL)`	Managed by Mastra Memory library via LibSQLStore adapter
`Short-term (LibSQL)`	Scoped to sessionId (UUID) -- cleared when session is deleted
`Long-term (MongoDB)`	Structured memory items extracted by LLM from each turn
`Long-term (MongoDB)`	Stored in memory_items collection with embedding vectors
`Long-term (MongoDB)`	Retrieved by cosine similarity of current prompt vs stored embedd
`Long-term (MongoDB)`	Persists across sessions -- never auto-deleted

`session/memory.ts`

Short-term conversation memory -- LibSQL via Mastra Memory library

Key exported functions:

ensureThread(threadId, title, intent) -- creates thread in LibSQL if not exists, updates title/metadata if it already exists. Silent on failure.

getMemoryContext(threadId) -- retrieves last 6 messages via memory.getContext(), converts Mastra format to Vercel AI SDK CoreMessage[] format for LLM injection.

saveConversationTurn({ threadId, prompt, intent, assistant }) -- saves two messages (user + assistant) to LibSQL. Each message stores both plain text (for Mastra) and a structured uiMessage in metadata (for the history API to reconstruct conversations).

listSessions() / getSessionDetail(id) / deleteSession(id) -- session management API backed by Mastra's thread storage.

The RESOURCE_ID constant ("mindai") is used as a namespace in Mastra's storage so threads from different apps don't collide if sharing the same SQLite file.

session/memory.ts -- message storage format

```
// How a message is stored (simplified)
{
  id: randomUUID(),
  role: "user",
  threadId: sessionId,
  content: {
```

```
      format: 2,
      parts: [{ type: "text", text: "show spending by city" }],
      metadata: {
        uiMessage: { messageId, role: "user", prompt, intent, createdAt }
      }
    }
  }
```

session/agent.ts

Mastra Agent -- free-text routing when no intent is provided

Creates a Mastra Agent with the "supervisor" LLM model and 3 registered tools. Used only when the user sends a prompt without specifying an intent.

Tools:
buildDashboard -- calls features/dashboard.executeDashboard()
generateReport -- calls features/report.executeReport()
executeInquiry -- calls features/inquiry.executeInquiry()

The agent's instructions (system prompt) are read from skills/analytics/SKILL.md. MAX_AGENT_STEPS=2 limits how many tool calls the agent can make in one turn. analyticsAgent.generateLegacy() is called with the user message and returns the tool result from whichever tool the LLM chose.

memory/memory.schema.ts

Mongoose schema for long-term memory items

Collection: memory_items. Fields:

userId (indexed) -- scopes all memories to one user
sessionId -- which session the memory came from
type -- goal | insight | preference | context | decision
content -- the actual memory text (5-300 chars)
tags[] -- keyword strings for fallback retrieval
importance -- integer 1-5 (5 = most important, recalled first)
embedding[] -- 384 floats from all-MiniLM-L6-v2 (for semantic search)

Two compound indexes:
{ userId, importance DESC, createdAt DESC } -- for ranked listing
{ userId, tags } -- for tag-based fallback search

memory/memory.repository.ts

MongoDB CRUD for memories with semantic retrieval

upsert(items[]) -- for each item:
1. Generates embedding via generateEmbedding(content)
2. Checks for existing similar memory (first 60 chars fingerprint match)

3. Updates if exists (refreshes content, tags, importance, embedding)
4. Creates if new

findRelevant(userId, promptText, limit) -- semantic retrieval:

1. Loads up to 100 memories for the user (sorted by importance + recency)
2. Generates embedding of the current promptText
3. If model ready: scores each memory by cosine similarity
4. Fallback if model not ready: keyword overlap score + importance/20 bonus
5. Returns top N by score

listByUser() -- returns all memories without the embedding field (too large for API)
deleteByUser() -- clears all memories for a user

`memory/memory.service.ts`

Memory service -- getRelevantContext() + extractAndSave()

getRelevantContext(userId, prompt) -- called before every LLM call. Fetches the top 3 semantically relevant memories and formats them as:
[INSIGHT] User prefers budget breakdowns by municipality
[GOAL] Monitoring road infrastructure projects in northern regions
This string is injected into the conversation context as a synthetic "user" message before the real history, giving the Supervisor LLM long-term context.

extractAndSave(userId, sessionId, prompt, summary, apiKey) -- fires after every response. Calls extractMemories() (LLM), then calls repo.upsert(). Any failure is caught and logged as a warning -- never blocks the response.

CHAPTER 9

Remaining Modules

Cache, History, Sources, User Keys, Saved Results, Prompts

Cache Module

cache/cache.service.ts

Injectable prompt cache -- SHA-256 keyed, 7-day TTL in MongoDB

cacheKey(intent, prompt) -- generates a 24-char SHA-256 prefix from "intent:lowercased_normalized_prompt". Minor whitespace differences in prompts still hit the same cache entry.

getCached<T>(intent, prompt) -- findOneAndUpdate() with $inc on hitCount and $set on lastHitAt. Returns the result field cast to T, or null on miss.

setCached<T>(intent, prompt, result) -- replaceOne with upsert:true. Overwrites if key exists. The TTL is enforced by a MongoDB TTL index on createdAt (7 days = 604800 seconds).

list() -- returns all cache entries (without result field -- too large) sorted by lastHitAt.
deleteEntry(key) / clearAll() -- cache management API endpoints.

cache/prompt-cache.ts

Standalone cache functions -- used only by Mastra agent fallback path

Identical logic to CacheService but implemented as plain async functions using mongoose.connection.db (the raw connection, not @InjectModel). Used by features/pipeline.ts which is called from session/agent.ts (Mastra tools are not inside NestJS DI, so they cannot inject CacheService).

History Module

history/results-history.repository.ts

Saves and queries pipeline run history in MongoDB

Collection: results_history (via PipelineRun Mongoose schema).

save(entry) -- creates a document with prompt, intent, collection, pipeline stages, rows, rowCount, and durationMs. Returns the MongoDB _id as a hex string.

list(filter) -- paginated query with optional intent filter. Max 100 results per page. Sorted by createdAt DESC.

findById(id) -- validates the id is a valid 24-char MongoDB ObjectId hex before querying.

count(filter) -- used to return total count alongside paginated results.

history/history.service.ts + history.controller.ts

API layer for history and sessions

HistoryService wraps the repository and also delegates to session/memory.ts for session-related operations. This keeps the controller clean.

Routes:

```
GET /api/history/results          -- list runs (pagination: skip, limit, intent filter)
GET /api/history/results/:id      -- full detail (includes pipeline stages and rows)
GET /api/history/sessions         -- list all conversation sessions
GET /api/history/sessions/:id     -- one session with full message history
DELETE /api/history/sessions/:id  -- permanently delete a session
```

Sources Module

sources/sources-cache.ts + source.repository.ts

In-memory source cache and normalizeToken()

sources-cache.ts: A module-level DataSource[] array. getSources() and setSourcesCache() are the only API. Zero MongoDB hits per request -- the array is loaded once at startup and updated synchronously whenever a source is added/removed.

source.repository.ts: contains normalizeToken(s) -- converts strings to lowercase alphanumeric. This lets "My Source", "my_source", "MY SOURCE" all match the same registered collection.

sources/sources.service.ts

Source registry -- loads on startup, CRUD, generates example prompts

Implements OnModuleInit -- calls reloadCache() on app startup so sources are available before the first HTTP request.

getMeta() -- for each registered source, generates example prompts for all 3 intents. Used by GET /api/meta (no auth required) so the frontend can show prompt suggestions.

register(source) -- upsert by collection name, then reload cache.
remove(collection) -- delete by collection, reload cache.

User Keys Module

user-keys/user-keys.service.ts

Stores per-user Groq API keys in MongoDB

Collection: user_api_keys. One document per userId, with apiKey field.

save(userId, apiKey) -- findOneAndUpdate with upsert:true.
get(userId) -- returns the stored key or null.
delete(userId) -- removes the document. Called automatically by AnalyticsService when a user's key is rejected by Groq (invalid key).

Saved Results Module

saved-results/*.ts

User-saved report/dashboard snapshots

Allows users to bookmark specific analytics results. All operations are scoped to userId.

Routes (via SavedResultsController):
GET /api/saved-results -- list user's saved items
GET /api/saved-results/:id -- one saved item
POST /api/saved-results -- save a result
DELETE /api/saved-results/:id -- remove a saved result

Prompts

prompts/planner.prompt.ts

Builds Supervisor system prompt with full schema injected

buildPlannerPrompt(intent, sources) -- generates the system prompt for LLM Call #1. It injects the name, description, collection name, and field definitions of every registered DataSource. This is how the LLM knows what data is available to query. The prompt also specifies the required output format, forbidden pipeline stages, and intent-specific instructions (e.g. for dashboard: include strategy and chartHint).

prompts/chart.prompt.ts

Builds chart LLM message with data sample

buildChartPrompt(rows, prompt, strategy, chartHint, source) -- builds the user message for LLM Call #2 (chart). Includes: the first CHART_SAMPLE_ROWS rows as JSON, field types inferred from the data, available chart types, the layout options, the strategy hint from the Supervisor, and the original prompt. All available in one message so the LLM can make informed widget choices.

prompts/writer.prompt.ts

Builds messages for inquiry and report LLM calls

buildInquiryMessage(prompt, rows, maxRows, maxChars) -- formats data rows as text (truncated to maxChars) and adds the original question. buildReportMessage(prompt, rows, maxChars, withChart) -- same but includes a hint about whether a chart accompanies the report so the writer can adjust prose.

Types

types/index.ts

All shared TypeScript interfaces

Type / Interface	Description
DataSource	A registered data collection: name, collection, description, fiel
DataSourceField	One field: name, type, description, enum values, referencing info
DataSourceJoin	Cross-collection join definition
IntentKind	"dashboard" | "report" | "general_question"
SkillKind	"aggregation" | "chart" | "report" | "inquiry"
TaskPlan	LLM #1 output: needsData, pipeline[], skills[], strategy, chartHi
DashboardSpec	Final dashboard: layout, title, summary, widgets[]
WidgetSpec	One chart widget: id, type, title, insight, option (ECharts)
WidgetType	All 15 chart type strings (bar_chart, donut_chart, etc.)
FieldType	string|number|integer|date|enum|reference|array|object|geo|text
ChartHint	Hint string from supervisor to guide chart selection

CHAPTER 10

Environment Variables

Complete reference for all configuration options

Required Variables

Variable	Description
MONGODB_URI	MongoDB connection string e.g. mongodb://127.0.0.1:27017/mind_pla
MONGODB_DB	Database name e.g. mind_platform

LLM Configuration

Variable	Default / Description
GROQ_API_KEY	Global Groq API key (optional if all users provide their own)
GROQ_MODEL	llama-3.3-70b-versatile -- default for all roles
GROQ_SUPERVISOR_MODEL	Override model for the Supervisor role
GROQ_CHART_MODEL	Override model for the Chart role
GROQ_WRITER_MODEL	Override model for the Writer role
GROQ_MEMORY_MODEL	Override model for memory extraction (default: llama-3.1-8b-insta
SUPERVISOR_TIMEOUT_MS	8000 -- abort supervisor LLM call after N ms
CHART_TIMEOUT_MS	8000 -- abort chart LLM call after N ms
WRITER_TIMEOUT_MS	8000 -- abort writer/memory LLM call after N ms

Server Configuration

Variable	Default / Description
PORT	3001 -- HTTP port for the NestJS app
NODE_ENV	production | development
ALLOWED_ORIGINS	Comma-separated CORS origins (empty = allow all)
API_KEY	Optional server-level auth key (x-api-key header)
SHUTDOWN_TIMEOUT_MS	10000 -- graceful shutdown window

Tuning Variables (optional)

Variable	Default / Description
MONGODB_SERVER_SELECTION_TIMEOUT_MS	8000 -- MongoDB connection timeout
MONGODB_CONNECT_RETRIES	1 -- how many times to retry connecting
MONGODB_PIPELINE_TIMEOUT_MS	30000 -- max time for MongoDB aggregation

CHART_MAX_WIDGETS	3 -- max widgets per dashboard
CHART_MAX_TOKENS	1000 -- token limit for chart LLM call
CHART_SAMPLE_ROWS	8 -- rows sent to chart LLM as sample
INQUIRY_MAX_TOKENS	400 -- token limit for inquiry responses
INQUIRY_MAX_ROWS	10 -- rows sent to inquiry writer
REPORT_MAX_TOKENS	900 -- token limit for report responses
WRITER_MAX_CHARS	8000 -- max data characters sent to writer
PLANNER_MAX_TOKENS	600 -- token limit for supervisor plan
LIBSQL_URL	file:./data/memory.db -- LibSQL connection
TRACE	1 -- enable verbose JSON dumps in logs
DEBUG	1 -- enable debug log level

CHAPTER 11

Key Design Decisions

Why the code is structured the way it is

Decision	Reason
`Hybrid LLM + Deterministic charts`	LLM picks chart types; pure code renders them. Avoids hallucinate
`In-memory source cache`	Zero DB reads per request for schema. Sources rarely change, load
`7-day prompt cache`	Identical prompts skip both LLM calls. SHA-256 key + normalized p
`Global APP_GUARD`	Security check cannot be forgotten on new routes -- it applies by
`AsyncLocalStorage for request ID`	No need to thread requestId through every function parameter.
`Fire-and-forget for memory/history`	Memory extraction and history logging never delay the HTTP respon
`API key fallback mechanism`	Users with expired keys still work if a global key is configured.
`Forbidden pipeline stages`	$merge/$out/$function blocked to prevent data corruption or RCE.
`Local embedding model (WASM)`	No API cost, no external dependency, works offline. 25MB one-time
`Memory extracted by small model`	llama-3.1-8b-instant is fast/cheap for extraction. 70B model rese
`Two memory systems`	LibSQL for short-term (fast, local), MongoDB for long-term (persi
`Two pipeline paths (DI + standalone)`	NestJS DI path for normal requests; standalone functions for Mast